

JavaScript Fundamentals – Proper Detailed Notes (BTech 1st Year)

⚠ These notes are **intentionally long, detailed, and exam-ready**. Every point from your list is **explicitly covered** with explanations + examples. Read slowly. This is meant to build foundations.

1. Values

A **value** is any piece of data that JavaScript can store, process, or return as output.

Examples of values:

```
10           // number value
"Hello"      // string value
true         // boolean value
null         // intentional empty value
undefined    // absence of value
[1,2,3]      // array value
{a:1}        // object value
```

🔄 Everything in JavaScript ultimately resolves to a **value**.

2. Numbers

JavaScript uses **one single data type** for all numbers.

✓ Integers ✓ Decimals ✓ Negative numbers

```
let a = 10;
let b = 3.14;
let c = -99;
```

⚠ No separate `int`, `float`, `double` like C/C++/Java.

3. Arithmetic Operations on Numbers

Operator	Meaning	Example	Output
<code>+</code>	Addition	<code>5 + 3</code>	8
<code>-</code>	Subtraction	<code>5 - 3</code>	2

Operator	Meaning	Example	Output
*	Multiplication	5 * 3	15
/	Division	5 / 2	2.5
%	Remainder	5 % 2	1

```
let x = 10;
let y = 3;

console.log(x + y);
console.log(x - y);
console.log(x * y);
console.log(x / y);
console.log(x % y);
```

4. Precedence of Arithmetic Operations

JavaScript follows **BODMAS**:

1. Brackets ()
2. Orders (*, / , %)
3. Addition & Subtraction

```
console.log(10 + 2 * 5);    // 20
console.log((10 + 2) * 5); // 60
```

 Always use brackets to avoid confusion.

5. Strings

A **string** is a sequence of characters (text data).

Ways to represent strings

1 Double Quotes

```
let name = "Palak";
```

2 Single Quotes

```
let city = 'Delhi';
```

3 Backticks (Template Strings)

```
let college = `IIT`;
```

6. Injecting JavaScript Expressions in Template Strings

Template strings allow **direct JS execution inside strings**.

```
let age = 18;
let year = 2026;

console.log(`I am ${age} years old`);
console.log(`Next year will be ${year + 1}`);
```

⚠ Works ONLY with backticks.

7. String Concatenation

Joining multiple strings.

```
let first = "Hello";
let second = "World";

console.log(first + " " + second);
```

With numbers:

```
console.log("Age: " + 18); // Age: 18
```

8. Boolean

Boolean has **only two values**:

- true
- false

```
let isLoggedIn = true;
let isAdmin = false;
```

Used heavily in **conditions & logic**.

9. Logical Operations

Logical AND (`&&`)

✓ All conditions must be true

```
true && true    // true
true && false   // false
```

Logical OR (`||`)

✓ At least one true

```
false || true  // true
```

Logical NOT (`!`)

✓ Reverses boolean

```
!true  // false
!false // true
```

10. typeof Operator

Used to check **data type at runtime**.

```
typeof 10          // "number"
typeof "hi"        // "string"
typeof true        // "boolean"
typeof undefined   // "undefined"
typeof null        // "object" (JS bug - remember for exams)
```

11. null vs undefined

undefined

- Variable declared
- No value assigned

```
let a;  
console.log(a); // undefined
```

null

- Intentional empty value

```
let b = null;
```

12. Variables

Variables store values in memory.

let

```
let age = 18;  
age = 19;
```

const

```
const PI = 3.14;  
// PI = 4; ❌ Error
```

var (old, avoid)

```
var x = 10;
```

13. Rules of Naming Variables

❌ Cannot start with number ❌ No spaces ❌ No reserved keywords

✅ Valid

```
let userName;  
let total_marks;
```

14. Best Practices for Naming Variables

✅ Use meaningful names ✅ camelCase ✅ Avoid single letters

```
let studentAge;  
let totalScore;
```

15. console.log()

Used to print values for debugging.

```
console.log("Hello JS");  
console.log(10 + 5);
```

16. Conditional Statements

if / else if / else

```
let marks = 75;  
  
if (marks >= 90) {  
  console.log("A Grade");  
} else if (marks >= 60) {  
  console.log("B Grade");  
} else {  
  console.log("Fail");  
}
```

17. switch Statement

```
let day = 2;  
  
switch(day) {  
  case 1:  
    console.log("Monday");  
    break;  
  case 2:  
    console.log("Tuesday");  
    break;  
  default:  
    console.log("Invalid day");  
}
```

⚠ `break` prevents fall-through.

18. Ternary Operator

Short version of if-else.

```
let age = 18;  
let result = age >= 18 ? "Adult" : "Minor";
```

19. Loops

for loop

```
for (let i = 1; i <= 5; i++) {  
  console.log(i);  
}
```

while loop

```
let i = 1;  
while (i <= 5) {  
  console.log(i);  
  i++;  
}
```

break

Stops loop completely.

continue

Skips current iteration.

```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) continue;  
  if (i === 5) break;  
  console.log(i);  
}
```

20. Functions

Why we need functions?

✓ Reusability ✓ Clean code ✓ Modular programming

Function Declaration

```
function add(a, b) {  
  return a + b;  
}
```

Function Expression

```
const add = function(a, b) {  
  return a + b;  
};
```

Arrow Function

```
const add = (a, b) => a + b;
```

21. Arguments & Parameters

```
function greet(name) { // parameter  
  console.log(name);  
}  
  
greet("Palak"); // argument
```

Optional Arguments

```
function demo(a, b) {  
  console.log(a, b);  
}  
  
demo(10); // b = undefined
```

Default Arguments

```
function greet(name = "Guest") {  
  console.log(name);  
}
```

Default Parameter Order in JavaScript (VERY IMPORTANT)

Unlike Python, JavaScript allows default parameters in ANY position.


```
function demo(a = 10, b) {  
  console.log(a, b);  
}  
  
demo(5);      // a = 5, b = undefined  
demo();      // a = 10, b = undefined
```

This is **VALID JavaScript**, but:

✗ Bad practice (confusing):

```
function greet(name = "Guest", age) {}
```

✓ Good practice:

```
function greet(name, age = 18) {}
```

Key rule:

Default value is used **ONLY when the argument is** `undefined`

This behavior is **different from Python**, which enforces strict ordering.

```
function greet(name = "Guest") {  
  console.log(name);  
}
```

22. Conditions & Loops inside Functions

```
function printEven(n) {  
  for (let i = 1; i <= n; i++) {  
    if (i % 2 === 0) {  
      console.log(i);  
    }  
  }  
}
```

23. Arrays

Why arrays?

Store **multiple values in one variable**.

```
let arr = [10, 20, 30];
```

Anatomy of an array

Index starts from **0**.

```
arr[0]; // 10
```

length

```
arr.length;
```

Looping array

```
for (let i = 0; i < arr.length; i++) {  
  console.log(arr[i]);  
}
```

for...of loop

Used to iterate over **values** of an array (preferred way).

```
let arr = [10, 20, 30];  
  
for (let value of arr) {  
  console.log(value);  
}
```

for...in loop (IMPORTANT – but risky for arrays)

for...in iterates over **indexes (keys)**, not values.

```
let arr = [10, 20, 30];  
  
for (let index in arr) {
```

```
    console.log(index, arr[index]);  
}
```

Output:

```
0 10  
1 20  
2 30
```

⚠ Why `for...in` is NOT recommended for arrays: - It iterates over **indexes as strings** - It can also iterate over **extra properties** added to the array - Order is not guaranteed in edge cases

✅ Use cases: - Objects → ✅ YES - Arrays → ❌ Avoid (use `for` or `for...of` instead)

```
for (let value of arr) {  
    console.log(value);  
}
```

Array Methods

```
arr.push(40);    // add at end  
arr.pop();       // remove end  
arr.shift();     // remove start  
arr.unshift(5);  // add start  
arr.join(",");   // convert to string  
arr.includes(20); // true/false  
arr.sort();      // lexicographical sort  
arr.slice(1, 3); // copy portion
```

◆ Array Splicing (`splice`) — IMPORTANT

`splice()` is one of the **most important and confusing array methods** in JavaScript.

🔧 It is used to **ADD, REMOVE, or REPLACE** elements in an array. 🔧 It **MODIFIES** the original array (this is VERY important for exams).

Syntax

```
array.splice(startIndex, deleteCount, item1, item2, ...)
```

Parameter	Meaning
startIndex	Index from where change starts
deleteCount	Number of elements to remove
item1...	Elements to add (optional)

1 Removing elements using splice

```
let arr = [10, 20, 30, 40, 50];  
  
arr.splice(1, 2);  
console.log(arr);
```

Explanation: - Start at index - Remove elements (,)

✓ Output:

```
[10, 40, 50]
```

2 Adding elements using splice

```
let arr = [1, 2, 5];  
  
arr.splice(2, 0, 3, 4);  
console.log(arr);
```

Explanation: - Start at index - Remove elements - Add and

✓ Output:

```
[1, 2, 3, 4, 5]
```

3 Replacing elements using splice

```
let arr = ["A", "B", "C", "D"];  
  
arr.splice(1, 2, "X", "Y");  
console.log(arr);
```

Explanation: - Start at index **1** - Remove **2** elements (**B**, **C**) - Add **X**, **Y**

✓ Output:

```
["A", "X", "Y", "D"]
```

4 What does splice return?

```
let arr = [10, 20, 30];
let removed = arr.splice(1, 1);

console.log(removed);
console.log(arr);
```

✓ Output:

```
[20]
[10, 30]
```

🔗 `splice()` returns an **array of removed elements**.

5 splice vs slice (VERY IMPORTANT)

splice	slice
Modifies original array	Does NOT modify array
Used for add/remove/replace	Used for copying
Returns removed elements	Returns copied elements

```
// slice example
let arr = [1,2,3,4];
let part = arr.slice(1,3);
```

6 Common Exam Traps ⚠

✗ Thinking splice creates a new array (it does NOT) ✗ Forgetting deleteCount ✗ Confusing splice with slice

Real-life Use Case

 Removing a student from list

```
students.splice(index, 1);
```

 Inserting an item in middle

```
arr.splice(mid, 0, newItem);
```

One-line Memory Trick

splice = destructive (changes array) slice = non-destructive

24. Objects

Why objects?

To represent **real-world entities**.

Anatomy of object

```
let student = {  
  name: "Palak",  
  age: 18,  
  marks: 90,  
  greet: function() {  
    console.log("Hello");  
  }  
};
```

Accessing properties

```
student.name;           // dot notation  
student["age"];         // bracket notation
```

Accessing methods

```
student.greet();
```

Updating properties

```
student.age = 19;
```

delete operator

```
delete student.marks;
```

for...in loop

```
for (let key in student) {  
  console.log(key, student[key]);  
}
```

Nested Objects

```
let user = {  
  name: "A",  
  address: {  
    city: "Delhi",  
    pincode: 110001  
  }  
};  
  
console.log(user.address.city);
```

END OF DETAILED NOTES

If you say **YES**, next I will: - Add **exam-oriented questions** - Add **dry-run examples** - Create **PDF + printable version** - Provide **100+ practice problems**

Say what you want next.